

```
1 using System;
2 using System.Threading;
3 using Microsoft.Data.SqlClient;
4 using S7.Net;
5
6 namespace Gateway
7 {
8     internal class Program
9     {
10         // ===== PLC (HANDLING) =====
11         static readonly string PlcIp = "192.168.0.201";
12         static readonly CpuType PlcCpu = CpuType.S71500;
13         static readonly short PlcRack = 0;
14         static readonly short PlcSlot = 1;
15
16         // ===== PLC (ROBOT QUALITY) =====
17         static readonly string RobotPlcIp = "192.168.0.211";
18         static readonly CpuType RobotPlcCpu = CpuType.S71500;
19         static readonly short RobotPlcRack = 0;
20         static readonly short RobotPlcSlot = 1;
21
22         // ===== HANDLING PLC DB =====
23         const string DB = "DB5";
24
25         // Handshake bits
26         const string NewOrderAddr = DB + ".DBX0.0";
27         const string OrderAckAddr = DB + ".DBX0.1";
28
29         // Order data (DB5)
30         const string OrderIdAddr = DB + ".DBD2";
31         const string RedQtyAddr = DB + ".DBW6";
32         const string BlackQtyAddr = DB + ".DBW8";
33         const string TotalQtyAddr = DB + ".DBW10";
34         const int CupArrayStartDbw = 12;
35
36         // ===== HANDLING SIGNALS =====
37         const string StartOrderSignal = "DB15.DBX0.1";
38         const string OrderDoneSignal = "M34.2";
39
40         // ===== ROBOT QUALITY DB =====
41         const int QualityDbNumber = 15;
42
43         const int QcOrderIdOffset = 0;           // Byte
44         const int QcJarIdOffset = 1;             // Byte
45         const int QcWeightDifferenceOffset = 2;   // REAL
46         const int QcMinutesOffset = 6;           // INT
47         const int QcSecondsOffset = 8;           // INT
48         const int QcRealWeightOffset = 10;        // REAL
49
50         // ===== ROBOT ToHMI DB (DB22) =====
51         const int ToHmiDbNumber = 22;
52         const int JarsOkOffset = 2;               // INT, DBW2
53         const int JarsFaultyOffset = 4;           // INT, DBW4
```

```
54
55 // ===== FINISH DELAY =====
56 static readonly TimeSpan FinishDelay = TimeSpan.FromSeconds(2);
57
58 // ===== SQL =====
59 static readonly string SqlConnStr =
60     "Server=localhost\
        \SQLEXPRESS;Database=ERP_DB;Trusted_Connection=True;TrustS
        erverCertificate=True;";
61
62 // Edge detection memory
63 static bool _lastStartOrder = false;
64 static bool _lastOrderDone = false;
65
66 // Current order being handled
67 static int? _currentOrderId = null;
68
69 // Delayed finish state
70 static int? _finishPendingOrderId = null;
71 static DateTime? _finishPendingSince = null;
72 static DateTime? _finishDetectedAt = null;
73
74 // Last robot values to avoid duplicate writes
75 static int _lastQcOrderId = -1;
76 static int _lastQcJarId = -1;
77 static float _lastQcWeightDifference = float.MinValue;
78 static float _lastQcRealWeight = float.MinValue;
79 static short _lastQcMinutes = short.MinValue;
80 static short _lastQcSeconds = short.MinValue;
81
82
83 static int _lastFaultOrderId = -1;
84 static int _lastJarsFaulty = int.MinValue;
85
86
87 static int _lastProducedOrderId = -1;
88 static int _lastJarsOk = int.MinValue;
89
90
91 static int _lastDeletedOrderWarned = -1;
92
93 static void Main(string[] args)
94 {
95     Console.WriteLine("=== ERP Gateway ===");
96     Console.WriteLine($"Handling PLC: {PlcIp}");
97     Console.WriteLine($"Robot PLC: {RobotPlcIp}");
98
99     using var plc = new Plc(PlcCpu, PlcIp, PlcRack, PlcSlot);
100     using var robotPlc = new Plc(RobotPlcCpu, RobotPlcIp,
        RobotPlcRack, RobotPlcSlot);
101
102     SafeWriteBool(plc, NewOrderAddr, false);
103     _currentOrderId = null;
```

```
104
105     while (true)
106     {
107         try
108         {
109             EnsurePlcConnected(plc, PlcIp);
110             EnsurePlcConnected(robotPlc, RobotPlcIp);
111
112             CheckIfKnownOrderWasDeleted();
113             HandleStartAndDoneSignals(plc);
114             HandleRobotQuality(robotPlc);
115             ProcessPendingFinish();
116             HandleNewOrderHandshake(plc);
117
118             Thread.Sleep(200);
119         }
120         catch (Exception ex)
121         {
122             Console.WriteLine($"[{Now()}] Gateway error:      ↗
123                 {ex.Message}");
124             Thread.Sleep(1000);
125         }
126     }
127
128     // ===== SEND ORDER TO HANDLING PLC      ↗
129     =====
130
131     static void HandleNewOrderHandshake(Plc plc)
132     {
133         if (_currentOrderId != null)
134             return;
135
136         if (_finishPendingOrderId != null)
137             return;
138
139         var order = GetNextPendingOrder();
140         if (order == null)
141         {
142             SafeWriteBool(plc, NewOrderAddr, false);
143             return;
144         }
145
146         WriteOrderToPlc(plc, order);
147
148         bool ack = ReadBool(plc, OrderAckAddr);
149
150         if (!ack)
151         {
152             SafeWriteBool(plc, NewOrderAddr, true);
153             Console.WriteLine($"[{Now()}] Sending OrderId=      ↗
154                 {order.OrderId} to PLC... waiting for      ↗
155                 OrderAck=TRUE");
```

```
153         return;
154     }
155
156     SafeWriteBool(plc, NewOrderAddr, false);
157     UpdateOrderStatus(order.OrderId, "Ready");
158     _currentOrderId = order.OrderId;
159
160     Console.WriteLine($"{Now()} PLC ack received. SQL: Order {order.OrderId} -> Ready");
161 }
162
163 // ===== START / DONE SIGNALS =====
164
165 static void HandleStartAndDoneSignals(Plc plc)
166 {
167     bool startNow = ReadBool(plc, StartOrderSignal);
168     bool doneNow = ReadBool(plc, OrderDoneSignal);
169
170     if (startNow && !_lastStartOrder)
171     {
172         int? id = _currentOrderId ?? FindLatestReadyOrderId() ?? FindLatestRunningOrderId();
173
174         if (id != null)
175         {
176             UpdateOrderStatus(id.Value, "Running");
177             SetStartedDateIfNull(id.Value, DateTime.Now);
178             Console.WriteLine($"{Now()} SQL: Order {id.Value} -> Running + StartedDate set");
179         }
180         else
181         {
182             Console.WriteLine($"{Now()} StartOrder TRUE but no Ready/Running order found.");
183         }
184     }
185
186     if (doneNow && !_lastOrderDone)
187     {
188         int? id = _currentOrderId ?? FindLatestRunningOrderId();
189
190         if (id != null)
191         {
192             StartPendingFinish(id.Value, DateTime.Now);
193         }
194         else
195         {
196             Console.WriteLine($"{Now()} OrderDone TRUE but no Running order found.");
197         }
198     }
199 }
```

```
200     _lastStartOrder = startNow;
201     _lastOrderDone = doneNow;
202 }
203
204 static void StartPendingFinish(int orderId, DateTime detectedAt)
205 {
206     if (_finishPendingOrderId == orderId)
207         return;
208
209     _finishPendingOrderId = orderId;
210     _finishPendingSince = DateTime.Now;
211     _finishDetectedAt = detectedAt;
212
213     Console.WriteLine($"[{Now()}] Order {orderId} marked for
        finish. Waiting {FinishDelay.TotalSeconds:0} seconds for
        final PLC updates...");
214 }
215
216 static void ProcessPendingFinish()
217 {
218     if (_finishPendingOrderId == null || _
        finishPendingSince == null || _finishDetectedAt == null)
219         return;
220
221     TimeSpan elapsed = DateTime.Now -
        _finishPendingSince.Value;
222     if (elapsed < FinishDelay)
223         return;
224
225     int orderId = _finishPendingOrderId.Value;
226     DateTime actualFinishedAt = _finishDetectedAt.Value;
227
228     SetFinishedDateStatusAndTime(orderId, actualFinishedAt);
229     Console.WriteLine($"[{Now()}] SQL: Order {orderId} ->
        Finished + FinishedDate + TimeSpentMinutes (using actual
        OrderDone time)");
230
231     _finishPendingOrderId = null;
232     _finishPendingSince = null;
233     _finishDetectedAt = null;
234     _currentOrderId = null;
235     _lastDeletedOrderWarned = -1;
236 }
237
238 // ===== ROBOT QUALITY =====
239
240 static void HandleRobotQuality(Plc robotPlc)
241 {
242     try
243     {
244         var qc = ReadRobotQuality(robotPlc);
245         if (qc == null)
```

```
246         return;
247
248         if (qc.OrderId > 0 && qc.JarId > 0)
249         {
250             if (QualityChanged(qc))
251             {
252                 UpdateOrderCupQuality(
253                     qc.OrderId,
254                     qc.JarId,
255                     qc.WeightDifference,
256                     qc.RealWeight,
257                     qc.TimeSpentMinutes);
258
259                 RememberQuality(qc);
260
261                 Console.WriteLine(
262                     $"{Now()} Robot QC saved -> OrderId=
263                     {qc.OrderId}, JarId={qc.JarId},
264                     WeightDifference=
265                     {qc.WeightDifference:0.###}, RealWeight=
266                     {qc.RealWeight:0.###}, TimeSpentMinutes=
267                     {qc.TimeSpentMinutes:0.##}");
268             }
269         }
270
271         int? activeOrderId =
272             ResolveActiveOrderIdForProductionUpdate(qc.OrderId);
273
274         if (activeOrderId != null)
275         {
276             int jarsOk = ReadJarsOk(robotPlc);
277             if (ProducedQuantityChanged(activeOrderId.Value,
278                 jarsOk))
279             {
280                 UpdateOrderQuantityProduced
281                     (activeOrderId.Value, jarsOk);
282                 RememberProducedQuantity(activeOrderId.Value,
283                     jarsOk);
284
285                 Console.WriteLine(
286                     $"{Now()} Orders updated -> OrderId=
287                     {activeOrderId.Value}, QuantityProduced=
288                     {jarsOk}");
289             }
290
291             int jarsFaulty = ReadJarsFaulty(robotPlc);
292             if (FaultyQuantityChanged(activeOrderId.Value,
293                 jarsFaulty))
294             {
295                 UpdateOrderQuantityProductFault
296                     (activeOrderId.Value, jarsFaulty);
297                 RememberFaultyQuantity(activeOrderId.Value,
298                     jarsFaulty);
```

```
285
286         Console.WriteLine(
287             $"{Now()} Orders updated -> OrderId=
                {activeOrderId.Value},
                QuantityProductFault={jarsFaulty}");
288         }
289     }
290 }
291 catch (Exception ex)
292 {
293     Console.WriteLine($"{Now()} Robot QC read error:
        {ex.Message}");
294 }
295 }
296
297 static QualityDto? ReadRobotQuality(Plc robotPlc)
298 {
299     byte orderId = ReadByteDb(robotPlc, QualityDbNumber,
        QcOrderIdOffset);
300     byte jarId = ReadByteDb(robotPlc, QualityDbNumber,
        QcJarIdOffset);
301     float weightDifference = ReadRealDb(robotPlc,
        QualityDbNumber, QcWeightDifferenceOffset);
302     short minutes = ReadIntDb(robotPlc, QualityDbNumber,
        QcMinutesOffset);
303     short seconds = ReadIntDb(robotPlc, QualityDbNumber,
        QcSecondsOffset);
304     float realWeight = ReadRealDb(robotPlc, QualityDbNumber,
        QcRealWeightOffset);
305
306     double timeSpentMinutes = minutes + (seconds / 60.0);
307
308     return new QualityDto
309     {
310         OrderId = orderId,
311         JarId = jarId,
312         WeightDifference = weightDifference,
313         RealWeight = realWeight,
314         Minutes = minutes,
315         Seconds = seconds,
316         TimeSpentMinutes = Math.Round(timeSpentMinutes, 2)
317     };
318 }
319
320 static int ReadJarsOk(Plc robotPlc)
321 {
322     return ReadIntDb(robotPlc, ToHmiDbNumber, JarsOkOffset);
323 }
324
325 static int ReadJarsFaulty(Plc robotPlc)
326 {
327     return ReadIntDb(robotPlc, ToHmiDbNumber,
        JarsFaultyOffset);
```

```
328     }
329
330     static int? ResolveActiveOrderIdForProductionUpdate(int qcOrderId)
331     {
332         if (qcOrderId > 0 && IsOrderActiveForCounts(qcOrderId))
333             return qcOrderId;
334
335         if (_currentOrderId != null && IsOrderActiveForCounts
336             (_currentOrderId.Value))
337             return _currentOrderId.Value;
338
339         if (_finishPendingOrderId != null && IsOrderActiveForCounts
340             (_finishPendingOrderId.Value))
341             return _finishPendingOrderId.Value;
342
343         return FindLatestRunningOrderId();
344     }
345
346     static bool IsOrderActiveForCounts(int orderId)
347     {
348         if (_finishPendingOrderId == orderId)
349             return true;
350
351         return IsOrderRunning(orderId);
352     }
353
354     static bool QualityChanged(QualityDto qc)
355     {
356         if (qc.OrderId != _lastQcOrderId) return true;
357         if (qc.JarId != _lastQcJarId) return true;
358         if (Math.Abs(qc.WeightDifference - _
359             lastQcWeightDifference) > 0.0001f) return true;
360         if (Math.Abs(qc.RealWeight - _lastQcRealWeight) > 0.0001f)
361             return true;
362         if (qc.Minutes != _lastQcMinutes) return true;
363         if (qc.Seconds != _lastQcSeconds) return true;
364         return false;
365     }
366
367     static void RememberQuality(QualityDto qc)
368     {
369         _lastQcOrderId = qc.OrderId;
370         _lastQcJarId = qc.JarId;
371         _lastQcWeightDifference = qc.WeightDifference;
372         _lastQcRealWeight = qc.RealWeight;
373         _lastQcMinutes = qc.Minutes;
374         _lastQcSeconds = qc.Seconds;
375     }
376
377     static bool FaultyQuantityChanged(int orderId, int jarsFaulty)
378     {
379         if (orderId != _lastFaultOrderId) return true;
```



```
376         if (jarsFaulty != _lastJarsFaulty) return true;
377         return false;
378     }
379
380     static void RememberFaultyQuantity(int orderId, int jarsFaulty)
381     {
382         _lastFaultOrderId = orderId;
383         _lastJarsFaulty = jarsFaulty;
384     }
385
386     static bool ProducedQuantityChanged(int orderId, int jarsOk)
387     {
388         if (orderId != _lastProducedOrderId) return true;
389         if (jarsOk != _lastJarsOk) return true;
390         return false;
391     }
392
393     static void RememberProducedQuantity(int orderId, int jarsOk)
394     {
395         _lastProducedOrderId = orderId;
396         _lastJarsOk = jarsOk;
397     }
398
399     // ===== DELETE WARNING =====
400
401     static void CheckIfKnownOrderWasDeleted()
402     {
403         if (_currentOrderId != null && !OrderExists
404             (_currentOrderId.Value))
405         {
406             if (_lastDeletedOrderWarned != _currentOrderId.Value)
407             {
408                 Console.WriteLine($"[{Now()}] WARNING: Order
409                     {_currentOrderId.Value} was deleted from SQL.");
410                 _lastDeletedOrderWarned = _currentOrderId.Value;
411             }
412
413             _currentOrderId = null;
414             _finishPendingOrderId = null;
415             _finishPendingSince = null;
416             _finishDetectedAt = null;
417             return;
418         }
419
420         if (_finishPendingOrderId != null && !OrderExists
421             (_finishPendingOrderId.Value))
422         {
423             if (_lastDeletedOrderWarned !=
424                 _finishPendingOrderId.Value)
425             {
426                 Console.WriteLine($"[{Now()}] WARNING: Order
427                     {_finishPendingOrderId.Value} was deleted from
428                     SQL.");
429             }
430         }
431     }
432 }
```

```
423         _lastDeletedOrderWarned =  
            _finishPendingOrderId.Value;  
424     }  
425  
426     _finishPendingOrderId = null;  
427     _finishPendingSince = null;  
428     _finishDetectedAt = null;  
429 }  
430 }  
431  
432 // ===== LOAD CUPS =====  
433  
434 static int[] LoadCupWaterArray(int orderId)  
435 {  
436     var cups = new int[12];  
437  
438     using var conn = new SqlConnection(SqlConnStr);  
439     conn.Open();  
440  
441     using var cmd = new SqlCommand(@"  
442 SELECT TOP 12 CupNo, Color, WaterPercent  
443 FROM OrderCups  
444 WHERE OrderId = @orderId  
445 ORDER BY  
446     CASE WHEN Color = 'Red' THEN 0 ELSE 1 END,  
447     CupNo ASC;", conn);  
448  
449     cmd.Parameters.AddWithValue("@orderId", orderId);  
450  
451     using var r = cmd.ExecuteReader();  
452     int idx = 0;  
453     while (r.Read() && idx < 12)  
454     {  
455         int water = Convert.ToInt32(r["WaterPercent"]);  
456         cups[idx++] = water;  
457     }  
458  
459     return cups;  
460 }  
461  
462 // ===== PLC HELPERS =====  
463  
464 static void EnsurePlcConnected(Plc plc, string ip)  
465 {  
466     if (plc.IsConnected)  
467         return;  
468  
469     Console.WriteLine($"{Now()} Connecting PLC {ip}...");  
470     plc.Open();  
471  
472     if (!plc.IsConnected)  
473         throw new Exception($"PLC not connected: {ip}");  
474 }
```

```
475     Console.WriteLine($"{Now()} PLC connected: {ip}");
476 }
477
478 static bool ReadBool(Plc plc, string addr)
479 {
480     object? v = plc.Read(addr);
481     return v is bool b && b;
482 }
483
484 static byte ReadByteDb(Plc plc, int dbNumber, int byteOffset)
485 {
486     object? value = plc.Read(DataType.DataBlock, dbNumber, byteOffset, VarType.Byte, 1);
487     if (value == null) return 0;
488     return Convert.ToByte(value);
489 }
490
491 static short ReadIntDb(Plc plc, int dbNumber, int byteOffset)
492 {
493     object? value = plc.Read(DataType.DataBlock, dbNumber, byteOffset, VarType.Int, 1);
494     if (value == null) return 0;
495     return Convert.ToInt16(value);
496 }
497
498 static float ReadRealDb(Plc plc, int dbNumber, int byteOffset)
499 {
500     object? value = plc.Read(DataType.DataBlock, dbNumber, byteOffset, VarType.Real, 1);
501     if (value == null) return 0f;
502     return Convert.ToSingle(value);
503 }
504
505 static void SafeWriteBool(Plc plc, string addr, bool value)
506 {
507     try
508     {
509         if (!plc.IsConnected)
510             plc.Open();
511
512         plc.Write(addr, value);
513     }
514     catch
515     {
516     }
517 }
518
519 static void WriteOrderToPlc(Plc plc, OrderDto order)
520 {
521     plc.Write(OrderIdAddr, order.OrderId);
522     plc.Write(RedQtyAddr, (short)order.RedQty);
523     plc.Write(BlackQtyAddr, (short)order.BlackQty);
524     plc.Write(TotalQtyAddr, (short)order.TotalQty);
```

```
525
526         for (int i = 0; i < 12; i++)
527         {
528             int dbw = CupArrayStartDbw + (i * 2);
529             plc.Write($"{DB}.DBW{dbw}", (short)order.CupWater[i]);
530         }
531     }
532
533     // ===== SQL HELPERS =====
534
535     static OrderDto? GetNextPendingOrder()
536     {
537         using var conn = new SqlConnection(SqlConnStr);
538         conn.Open();
539
540         using var cmd = new SqlCommand(@"
541 SELECT TOP 1 OrderId, RedQty, BlackQty, TotalQty
542 FROM Orders
543 WHERE Status = 'Pending'
544 ORDER BY OrderId ASC;", conn);
545
546         using var r = cmd.ExecuteReader();
547         if (!r.Read())
548             return null;
549
550         int orderId = Convert.ToInt32(r["OrderId"]);
551         int red = Convert.ToInt32(r["RedQty"]);
552         int black = Convert.ToInt32(r["BlackQty"]);
553         int total = Convert.ToInt32(r["TotalQty"]);
554
555         int expectedTotal = red + black;
556         if (total != expectedTotal)
557             total = expectedTotal;
558
559         int[] water = LoadCupWaterArray(orderId);
560
561         return new OrderDto
562         {
563             OrderId = orderId,
564             RedQty = red,
565             BlackQty = black,
566             TotalQty = total,
567             CupWater = water
568         };
569     }
570
571     static void UpdateOrderStatus(int orderId, string status)
572     {
573         using var conn = new SqlConnection(SqlConnStr);
574         conn.Open();
575
576         using var cmd = new SqlCommand(@"
577 UPDATE Orders
```

```
578 SET Status = @status
579 WHERE OrderId = @id;", conn);
580
581         cmd.Parameters.AddWithValue("@status", status);
582         cmd.Parameters.AddWithValue("@id", orderId);
583
584         cmd.ExecuteNonQuery();
585     }
586
587     static void UpdateOrderCupQuality(int orderId, int jarId,
        float weightDifference, float realWeight, double
        timeSpentMinutes)
588     {
589         using var conn = new SqlConnection(SqlConnStr);
590         conn.Open();
591
592         using var cmd = new SqlCommand(@"
593 UPDATE OrderCups
594 SET WeightDifference = @WeightDifference,
595     RealWeight = @RealWeight,
596     TimeSpentMinutes = @TimeSpentMinutes
597 WHERE OrderId = @OrderId
598     AND CupNo = @CupNo;", conn);
599
600         cmd.Parameters.AddWithValue("@OrderId", orderId);
601         cmd.Parameters.AddWithValue("@CupNo", jarId);
602         cmd.Parameters.AddWithValue("@WeightDifference",
        weightDifference);
603         cmd.Parameters.AddWithValue("@RealWeight", realWeight);
604         cmd.Parameters.AddWithValue("@TimeSpentMinutes", Math.Round
        (timeSpentMinutes, 2));
605
606         int rows = cmd.ExecuteNonQuery();
607
608         if (rows == 0)
609         {
610             Console.WriteLine($"[{Now()}] WARNING: No OrderCups
        row updated for OrderId={orderId}, CupNo={jarId}");
611         }
612     }
613
614     static void UpdateOrderQuantityProduced(int orderId, int
        jarsOk)
615     {
616         using var conn = new SqlConnection(SqlConnStr);
617         conn.Open();
618
619         using var cmd = new SqlCommand(@"
620 UPDATE Orders
621 SET QuantityProduced = @QuantityProduced
622 WHERE OrderId = @OrderId;", conn);
623
624         cmd.Parameters.AddWithValue("@OrderId", orderId);
```

```
625         cmd.Parameters.AddWithValue("@QuantityProduced", jarsOk);
626
627         int rows = cmd.ExecuteNonQuery();
628
629         if (rows == 0)
630         {
631             Console.WriteLine($"[{Now()}] WARNING: No Orders row
632                 updated for OrderId={orderId}");
633         }
634
635         static void UpdateOrderQuantityProductFault(int orderId, int
636             jarsFaulty)
637         {
638             using var conn = new SqlConnection(SqlConnStr);
639             conn.Open();
640
641             using var cmd = new SqlCommand(@"
642 UPDATE Orders
643 SET QuantityProductFault = @QuantityProductFault
644 WHERE OrderId = @OrderId;", conn);
645
646             cmd.Parameters.AddWithValue("@OrderId", orderId);
647             cmd.Parameters.AddWithValue("@QuantityProductFault",
648                 jarsFaulty);
649
650             int rows = cmd.ExecuteNonQuery();
651
652             if (rows == 0)
653             {
654                 Console.WriteLine($"[{Now()}] WARNING: No Orders row
655                     updated for OrderId={orderId}");
656             }
657         }
658
659         static int? FindLatestRunningOrderId()
660         {
661             using var conn = new SqlConnection(SqlConnStr);
662             conn.Open();
663
664             using var cmd = new SqlCommand(@"
665 SELECT TOP 1 OrderId
666 FROM Orders
667 WHERE Status = 'Running'
668       AND FinishedDate IS NULL
669 ORDER BY COALESCE(StartDate, CreatedDate) DESC;", conn);
670
671             object? result = cmd.ExecuteScalar();
672             if (result == null || result == DBNull.Value)
673                 return null;
674
675             return Convert.ToInt32(result);
676         }
```

```
674
675     static int? FindLatestReadyOrderId()
676     {
677         using var conn = new SqlConnection(SqlConnStr);
678         conn.Open();
679
680         using var cmd = new SqlCommand(@"
681 SELECT TOP 1 OrderId
682 FROM Orders
683 WHERE Status = 'Ready'
684     AND FinishedDate IS NULL
685 ORDER BY CreatedDate DESC;", conn);
686
687         object? result = cmd.ExecuteScalar();
688         if (result == null || result == DBNull.Value)
689             return null;
690
691         return Convert.ToInt32(result);
692     }
693
694     static bool IsOrderRunning(int orderId)
695     {
696         using var conn = new SqlConnection(SqlConnStr);
697         conn.Open();
698
699         using var cmd = new SqlCommand(@"
700 SELECT COUNT(1)
701 FROM Orders
702 WHERE OrderId = @id
703     AND Status = 'Running'
704     AND FinishedDate IS NULL;", conn);
705
706         cmd.Parameters.AddWithValue("@id", orderId);
707
708         int count = Convert.ToInt32(cmd.ExecuteScalar());
709         return count > 0;
710     }
711
712     static void SetStartedDateIfNull(int orderId, DateTime started)
713     {
714         using var conn = new SqlConnection(SqlConnStr);
715         conn.Open();
716
717         using var cmd = new SqlCommand(@"
718 UPDATE Orders
719 SET StartedDate = @dt
720 WHERE OrderId = @id
721     AND StartedDate IS NULL;", conn);
722
723         cmd.Parameters.AddWithValue("@dt", started);
724         cmd.Parameters.AddWithValue("@id", orderId);
725
726         cmd.ExecuteNonQuery();
```

```
727     }
728
729     static void SetFinishedDateStatusAndTime(int orderId, DateTime finished)
730     {
731         using var conn = new SqlConnection(SqlConnStr);
732         conn.Open();
733
734         DateTime? startedDate = null;
735
736         using (var readCmd = new SqlCommand(@"
737 SELECT StartedDate
738 FROM Orders
739 WHERE OrderId = @id;", conn))
740         {
741             readCmd.Parameters.AddWithValue("@id", orderId);
742             object? result = readCmd.ExecuteScalar();
743
744             if (result != null && result != DBNull.Value)
745                 startedDate = Convert.ToDateTime(result);
746         }
747
748         double? timeSpentMinutes = null;
749         if (startedDate != null)
750             timeSpentMinutes = Math.Round((finished - startedDate.Value).TotalMinutes, 2);
751
752         using var cmd = new SqlCommand(@"
753 UPDATE Orders
754 SET FinishedDate = @dt,
755     Status = 'Finished',
756     TimeSpentMinutes = @TimeSpentMinutes
757 WHERE OrderId = @id;", conn);
758
759         cmd.Parameters.AddWithValue("@dt", finished);
760         cmd.Parameters.AddWithValue("@id", orderId);
761         cmd.Parameters.AddWithValue("@TimeSpentMinutes", (object?) timeSpentMinutes ?? DBNull.Value);
762
763         cmd.ExecuteNonQuery();
764     }
765
766     static bool OrderExists(int orderId)
767     {
768         using var conn = new SqlConnection(SqlConnStr);
769         conn.Open();
770
771         using var cmd = new SqlCommand(@"
772 SELECT COUNT(1)
773 FROM Orders
774 WHERE OrderId = @id;", conn);
775
776         cmd.Parameters.AddWithValue("@id", orderId);
```



```
777
778         int count = Convert.ToInt32(cmd.ExecuteScalar());
779         return count > 0;
780     }
781
782     static string Now()
783     {
784         return DateTime.Now.ToString("HH:mm:ss");
785     }
786
787     // ===== DTO =====
788
789     class OrderDto
790     {
791         public int OrderId { get; set; }
792         public int RedQty { get; set; }
793         public int BlackQty { get; set; }
794         public int TotalQty { get; set; }
795         public int[] CupWater { get; set; } = new int[12];
796     }
797
798     class QualityDto
799     {
800         public int OrderId { get; set; }
801         public int JarId { get; set; }
802         public float WeightDifference { get; set; }
803         public float RealWeight { get; set; }
804         public short Minutes { get; set; }
805         public short Seconds { get; set; }
806         public double TimeSpentMinutes { get; set; }
807     }
808 }
809 }
```